

# Midterm Revision, Chapters 1–4

Everything from logic levels to the NVIC, compressed. Read this after the eight weekly files, not instead of them

Covers: Dean Chapters 1–4. Exam is paper based and worth 35% of the lecture course.

Syllabus focus: logic, GPIO, architecture, and NVIC.

## 1 How to use this file

This one is shaped differently. There is no deep dive, because a deep dive is what the previous eight files were for. This is the compression pass: the whole examinable body of Chapters 1 to 4, organised by the kind of question it produces, with the traps flagged.

Work through it in three passes.

1. **The map** in Section 2, to check nothing is missing from your memory.
2. **The calculations** in Section 3. These are the questions with numeric answers, which means they are the questions you can lose marks on by arithmetic alone, and the ones you can bank by practising. Do them until they are boring.
3. **The comparisons and traps** in Sections 4 and 5. These are the short-answer marks.

The exam is paper based, so you will be writing register-level C by hand without a compiler to catch you. Section 6 is a set of code patterns worth being able to reproduce from memory.

## 2 The map of Chapters 1–4

| Ch | Weeks | What it examines                                                                                   |
|----|-------|----------------------------------------------------------------------------------------------------|
| 1  | 1     | Why embed a computer. The five attributes, the six constraints, the platform stack                 |
| 2  | 2–3   | Electrical limits of a pin. CMSIS, bit fields, clock gating, the pin mux, the GPIO registers, BSRR |
| 3  | 4–5   | Concurrency. Tasks, cooperative scheduling, response-time analysis, FSMs, hardware of-fload        |
| 4  | 6–8   | The CPU core, memory map, stack. Excep-tions, the interrupt chain, priority, data races            |

### The thread running through all four

Each chapter answers the same question at a different level: *how does software touch the physical world without lying about it?*

Chapter 1 says the world is analog and slow. Chapter 2 says a pin has electrical limits and

registers behind it. Chapter 3 says you cannot wait and watch at the same time. Chapter 4 says the hardware can interrupt you between any two instructions, and shared data pays for it.

If an exam question is open-ended, this is the arc to hang an answer on.

### 3 The calculations

Five kinds. Every one has appeared in the book's exercises.

#### 3.1 Current-limiting resistor

$$R = \frac{V_{DD} - V_F}{I_{LED}}$$

Worked, from the book:  $V_{DD} = 3.0$  V,  $I_{LED} = 4$  mA. Red LED  $V_F = 1.8$  V gives  $R = 1.2/0.004 = \mathbf{300\ \Omega}$ . Blue LED  $V_F = 2.7$  V gives  $R = 0.3/0.004 = \mathbf{75\ \Omega}$ .

Then check it against the limits: **25 mA** absolute max per pin, **8 mA** for the guaranteed  $V_{OL}$ , **150 mA** total for the device. Seven segments at 20 mA each is 140 mA, which passes every per-pin check and nearly fails the device check.

#### 3.2 Logic thresholds

Fractions of  $V_{DD}$ , never absolute volts. Logic 0 up to  $0.3V_{DD}$ , undefined between  $0.3V_{DD}$  and  $0.7V_{DD}$ , logic 1 from  $0.7V_{DD}$ . At 3.3 V: **0.99 V** and **2.31 V**.

#### 3.3 Bit field positions

MODERN occupies bits  $[2n+1 : 2n]$ , because the field is 2 bits wide. So PA5's mode is bits  $[11:10]$ , and its `_Pos` is 10.

Generally, for a field  $w$  bits wide indexed by pin  $n$ : position =  $n \times w$ . PUPDR and OSPEEDR are also 2 bits ( $w = 2$ ), AFR is 4 bits ( $w = 4$ , with the array split at pin 8), OTYPER is 1 bit ( $w = 1$ ).

#### 3.4 Response-time analysis

This is the Chapter 3 exercise format and it is the most likely long-form calculation. Given per-task execution times, find the worst-case delay from a switch press to the LED responding.

The method, every time:

1. Identify the **worst possible moment** for the event to arrive. Usually just after the longest task starts.
2. Add the remaining time in that task.
3. Add every task that runs before the one you care about, in scheduler order.
4. Add the time inside the target task until the visible change.

Worked, using the book's numbers for the task-based scheduler with Task\_Read\_Switches (1 ms), Task\_Flash (1 ms in RGB mode), Task\_RGB (1000 ms in RGB mode), running in that fixed order.

Press arrives just after Task\_RGB starts, so you wait almost the whole 1000 ms for it to finish. Then Task\_Read\_Switches runs (1 ms) and sets `g_flash_LED`. Then Task\_Flash runs and lights white. Worst case  $\approx 1000 + 1 = \mathbf{1001}$  ms, plus however far into Task\_Flash the LED write sits.

Now the same system with an interrupt reading the switches (0.01 ms) instead of a task. The press is recognised in 0.01 ms, but you still wait for Task\_RGB to finish before Task\_Flash can run:  $\approx 1000$  ms. **The interrupt fixed  $T_1$  and did nothing for  $T_2$ .**

**The two-part delay, and it is the point of the whole chapter**

$T_1$  = event to the shared variable being updated. Fixed by an **interrupt**.

$T_2$  = shared variable updated to the output changing. Fixed by **shorter tasks** (FSM), **priority**, or **preemption**.

Total response =  $T_1 + T_2$ . Almost every exercise in Chapter 3 is asking you to notice which of the two a given change improves. An interrupt alone leaves  $T_2$  untouched, which is why the book keeps going after introducing interrupts.

Third variant: FSM tasks, where Task\_RGB drops from 1000 ms to 334 ms because it now returns after one colour instead of three. Worst case  $\approx 334$  ms. Combine FSM with an interrupt and you get  $0.01 + 334$  ms. Add preemption and  $T_2$  collapses too.

Also asked: **the minimum time a switch must be held to be detected**. With polling that is the worst-case loop time. With an interrupt it is essentially zero, because the edge latches in EXTI\_PR and waits.

### 3.5 Interrupt latency and stack arithmetic

Latency = cycles  $\div$  clock frequency. Book's M0 at 48 MHz:  $16/48 \times 10^6 = \mathbf{333.3}$  ns. Your M4 at 72 MHz:  $12/72 \times 10^6 = \mathbf{167}$  ns.

Stack questions: the stack grows toward **smaller** addresses, and PUSH of  $n$  registers subtracts  $4n$  from SP. So PUSH {R0, R2} takes SP from 0x0000\_2220 to 0x0000\_2218. POP {R0-R7, PC} is nine registers, so SP goes from 0x0000\_2010 to 0x0000\_2034.

Exception entry pushes **eight** registers, so 32 bytes.

## 4 The comparisons

Short-answer marks live here. Each of these is a "distinguish X from Y" question.

|                              | First                                                                                    | Second                                                                 |
|------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Concurrency vs parallelism   | Activities progress over the same <i>period</i> . Achievable on one core by interleaving | Activities execute at the same <i>instant</i> . Impossible on one core |
| CPU vs MCU                   | Fetch, decode, execute hardware                                                          | CPU + peripherals + support circuits + usually memory, on one die      |
| HAL vs BSP                   | Generic, chip-level peripheral functions. Knows what a GPIO port is                      | Board-specific functions built on HAL. Knows which pin LED3 is on      |
| Polling vs event triggering  | Software repeatedly tests a condition                                                    | Task code runs only when hardware signals an event                     |
| Cooperative vs preemptive    | Tasks yield voluntarily when done. A long task delays everything                         | An urgent task can interrupt a running one                             |
| Push-pull vs open drain      | Drives both high and low, two transistors                                                | Pulls low or floats. Needed for shared buses like I <sup>2</sup> C     |
| Active high vs active low    | Anode to pin: output 1 lights it                                                         | Cathode to pin: output 0 lights it. Pins sink more than they source    |
| Thread vs Handler mode       | Normal execution, MSP or PSP                                                             | Servicing an exception, always MSP                                     |
| MSP vs PSP                   | Handlers and, by default, everything                                                     | Optional per-task stack when a kernel is present                       |
| ODR vs BSRR                  | Read-modify-write, three operations, can race                                            | Single atomic write, zeros ignored, no read needed                     |
| volatile vs critical section | Forces real memory access. Fixes <b>staleness</b>                                        | Prevents interruption. Fixes <b>races</b> and <b>partial updates</b>   |

And the definitions that get asked verbatim:

**Modularity:** how well a program groups related code and separates independent code.  
**Responsiveness:** how quickly a system responds to an input event. *Worst case*.  
**CPU overhead:** the fraction of time spent on code doing no useful application work.  
**Spaghetti code:** code that entangles unrelated features, complicating maintenance.  
**Clock gating:** disabling a circuit by blocking its clock, reducing power. A gated peripheral is *inert*, not idle.  
**Multiplexer:** an electronic selector switch routing one of  $N$  inputs to an output.  
**Data race:** ill-timed preemption of a critical section producing an incorrect result.  
**Critical section:** code that executes incorrectly unless it runs atomically.  
**Real-time system:** one in which tasks have deadlines, and missing one is a failure.  
**Finite state machine:** a state machine with all states and transitions defined.  
**Root function:** a task's main function, which may call others as subroutines.

## 5 The traps, collected

Every trap box from Weeks 1–8, in one list. If you remember nothing else, remember these, because they are where marks are lost and where lab time disappears.

### Chapter 1–2, electrical

- Logic thresholds are **fractions of  $V_{DD}$** , not fixed volts. A circuit fine at 3.0 V can fail at 3.3 V.
- **25 mA is an absolute maximum, not a design target.** 8 mA is where the voltage spec is guaranteed.
- **Per-pin limits and device limits are different questions.** Seven legal pins can break the 150 mA device budget.
- A white LED with  $V_F = 3.4$  V **cannot** run from 3.3 V. No resistor value fixes it.
- Drive strength **falls as  $V_{DD}$  falls**, so battery sag dims LEDs before anything else breaks.
- A floating input reads garbage. That is what pull resistors are for.

### Chapter 2–3, registers

- **Clock first, configure second.** A write to a gated peripheral vanishes silently. No fault, no warning.
- **Clock enable registers are  $|=$ , never  $=$ .** The book's own Listing 2.8 gets this wrong.
- $|=$  alone **cannot** set a 2-bit field to 01. OR only turns bits on. Clear the field first.
- Write 1UL, not 1. Shifting a signed int to bit 31 is undefined.
- Header names can differ from the reference manual (GPIOAEN vs IOPAEN). **The header wins.**
- Never remove volatile/\_IO from a register access.
- A dead peripheral pin: check port clock, then MODER, then AFSEL. In that order.

### Chapter 3, concurrency

- You **cannot fix the starter program by polling faster.** The delays are the feature. The problem is structural.
- A busy-wait loop needs volatile or the compiler deletes it.
- Busy-wait units are meaningless: they depend on clock, compiler, optimisation level, and flash wait states.
- HAL\_Delay has real millisecond units and **still blocks.**
- An interrupt fixes  $T_1$  only.  $T_2$  needs shorter tasks, priority, or preemption.
- Don't confuse **CPU overhead** with **CPU utilisation.** Busy doing useful work is healthy.

### Chapter 4, interrupts

- **Lower priority number = higher urgency.**
- Only **sixteen** EXTI lines, one per bit position, shared across all ports. **PA0 and PB0 cannot both interrupt. PA0 and PB1 can.**
- **SYSCFG needs its own APB2 clock.** Second-most-forgotten clock enable.
- EXTICR1..4 are accessed as EXTICR[0..3]. Line 13 is EXTICR[3].
- **Fail to clear EXTI\_PR and the interrupt fires forever.** Write 1 to clear, with  $=$  not  $|=$ .
- NVIC order: **set priority, clear pending, then enable.** Otherwise you take a spurious interrupt.
- Clearing *all* pending flags discards other lines' unserved requests.
- A misspelled handler name is **silently never called**; the weak default loops forever.

Symptom: hang.

- An **analog-mode** pin cannot generate EXTI, because its input buffer is disabled.
- **Never** `__enable_irq()` **unconditionally**. Save and restore PRIMASK.
- **volatile does not make a shared counter safe**. This is the most-missed point in the chapter.

## 6 Code patterns to reproduce from memory

Paper exam, no compiler. These four are the ones worth being able to write cold.

Listing 1: 1. Configure a pin as output and drive it. The four phases.

```
1 RCC->AHBENR |= RCC_AHBENR_GPIOAEN;           /* clock */
2 GPIOA->MODER &= ~GPIO_MODER_MODER5_Msk;      /* route: */
3 GPIOA->MODER |= (1UL << GPIO_MODER_MODER5_Pos); /* output */
4 GPIOA->BSRR = GPIO_BSRR_BS_5;                 /* drive high */
```

Listing 2: 2. Read-modify-write a multi-bit field.

```
1 REG &= ~FIELD_Msk;                          /* clear the field */
2 REG |= ((value << FIELD_Pos) & FIELD_Msk);   /* set, masked */
```

Listing 3: 3. The four-stage interrupt setup, condensed.

```
1 /* 1. GPIO */
2 RCC->AHBENR |= RCC_AHBENR_GPIOCEN;
3 MODIFY_FIELD(GPIOC->MODER, GPIO_MODER_MODER13, 0); /* input */
4 MODIFY_FIELD(GPIOC->PUPDR, GPIO_PUPDR_PUPDR13, 1); /* pull-up */
5 /* 2. SYSCFG: line 13 -> port C (value 2), register EXTICR[3] */
6 RCC->APB2ENR |= RCC_APB2ENR_SYSCFGCOMPEN;
7 MODIFY_FIELD(SYSCFG->EXTICR[3], SYSCFG_EXTICR4_EXTI13, 2);
8 /* 3. EXTI: unmask, both edges */
9 EXTI->IMR |= MASK(13);
10 EXTI->RTSR |= MASK(13);
11 EXTI->FTSR |= MASK(13);
12 /* 4. NVIC: priority, clear, enable -- in that order */
13 NVIC_SetPriority(EXTI4_15_IRQn, 3);
14 NVIC_ClearPendingIRQ(EXTI4_15_IRQn);
15 NVIC_EnableIRQ(EXTI4_15_IRQn);
```

Listing 4: 4. Handler and critical section.

```
1 volatile uint32_t g_counter;           /* volatile: shared with ISR */
2
3 void EXTI4_15_IRQHandler(void) {
4     if (EXTI->PR & MASK(13)) {
5         EXTI->PR = MASK(13);           /* write 1 to clear, first */
6         g_counter++;                   /* urgent work only */
7     }
8 }
9
10 void main_side(void) {
11     uint32_t s = __get_PRIMASK();      /* save */
12     __disable_irq();                   /* modify */
13     g_counter++;                       /* critical section */
14     __set_PRIMASK(s);                  /* restore */
15 }
```

## 7 A last pass: the numbers

**Electrical:** logic 0  $\leq 0.3V_{DD}$ , logic 1  $\geq 0.7V_{DD}$ ; at 3.3 V that is 0.99 V and 2.31 V.  $V_{OH} \geq V_{DD} - 0.4$ ,  $V_{OL} \leq 0.4$  V. Input current  $\leq 1 \mu\text{A}$ ,  $\approx 25 \text{ nA}$  at  $25^\circ\text{C}$ . Per pin 25 mA absolute / 8 mA guaranteed. Device total 150 mA. Supply 2.0–3.6 V.

**Registers:** GPIO base 0x4800\_0000. MODER 0x00, OTYPER 0x04, OSPEEDR 0x08, PUPDR 0x0C, IDR 0x10, ODR 0x14, BSRR 0x18, LCKR 0x1C, AFR 0x20/0x24, BRR 0x28. MODER: 00 in, 01 out, 10 alt, 11 analog. GPIO clock enables at AHBENR bits 17–22.

**Vectors:** 0x00 initial SP, 0x04 Reset, 0x08 NMI, 0x0C HardFault, 0x2C SVCALL, 0x38 PendSV, 0x3C SysTick, 0x40 IRQ0. **vector #** = 16+ **IRQ #**.

**Exceptions:** 8 registers stacked (xPSR, PC, LR, R12, R3–R0) = 32 bytes. Entry 16 cycles on M0, 12 on M4. LDM, STM, PUSH, POP, MULS abandoned and restarted. EXC\_RETURN: 0xFFFFFFFF1 Handler/MSP, 0xFFFFFFFF9 Thread/MSP, 0xFFFFFFFFD Thread/PSP.

**Priority:** M0 has 2 bits / 4 levels (raw 0, 64, 128, 192). Your F303 has 4 bits / 16 levels.

**Your board:** STM32F303VCT6, Cortex-M4 with FPU and DSP, 72 MHz max, 256 KB flash, 40 KB SRAM, up to 87 GPIO, 4 ADC, 2 DAC, ST-LINK/V2. Book's board: STM32F091RC, Cortex-M0, 48 MHz, 256 KB flash, 32 KB SRAM, no FPU, no DSP.

Good luck. Chapter 5 opens the compiler, and it is a much more relaxing read than this one.